# Conditional Latent Diffusion

Bora Turan, Ulaş Dabancı

January 6, 2026

## 1 Introduction

In this project, we moved beyond linear interpolation to explore Conditional Latent Diffusion. We focused on the task of Gender Swapping (transforming female images to male ones) using SDE-Edit.

## 2 Model Architecture & Training

Unlike standard diffusion models that operate on pixels, our model operates on 1D latent vectors ($1 \times 512$).

### 2.1 Latent Diffusion MLP

We designed a residual MLP that conditions on both the timestep $t$ and the target class label $c$.

Listing 1: Latent Diffusion MLP Architecture

```
class LatentDiffusionMLP(nn.Module):
    def __init__(self, input_dim=512, time_dim=64, hidden_dim=1024):
        super().__init__()

        #Time Embedding
        self.time_dim = time_dim
        self.time_mlp = nn.Sequential(
            nn.Linear(1, time_dim),
            nn.GELU(),
            nn.Linear(time_dim, time_dim),
        )

        #Condition Embedding, we map the single float label to a vector
        self.cond_mlp = nn.Sequential(
            nn.Linear(1, time_dim),
            nn.GELU(),
            nn.Linear(time_dim, time_dim),
        )

        #Main Network
        self.input_proj= nn.Linear(input_dim + time_dim + time_dim, hidden_dim)

        self.blocks = nn.Sequential(
            ResidualBlock(hidden_dim),
            ResidualBlock(hidden_dim),
            ResidualBlock(hidden_dim),
            ResidualBlock(hidden_dim),
            )

        self.output_proj = nn.Linear(hidden_dim, input_dim)
```

```python
    def forward(self, x, t, c):
        # x: [batch, 512] (Noisy Latent)
        # t: [batch, 1]   (Timestep)
        # c: [batch, 1]   (Condition/Label)

        t_emb = self.time_mlp(t)
        c_emb = self.cond_mlp(c)

        #We concatenate input, time, and condition along the feature dimension
        x_input = torch.cat([x, t_emb, c_emb], dim=1)

        h = self.input_proj(x_input)
        h = self.blocks(h)
        noise_pred = self.output_proj(h)

        return noise_pred
```

### 2.2 Training Results

The model was trained for 50 epochs using Mean Squared Error loss between the predicted noise and the actual added noise.

```
Epoch 5/50 | Loss: 0.16338
Epoch 10/50 | Loss: 0.15829
Epoch 15/50 | Loss: 0.15726
Epoch 20/50 | Loss: 0.15589
Epoch 25/50 | Loss: 0.15805
Epoch 30/50 | Loss: 0.16008
Epoch 35/50 | Loss: 0.15740
Epoch 40/50 | Loss: 0.15755
Epoch 45/50 | Loss: 0.15492
Epoch 50/50 | Loss: 0.15698
Training Complete!
```

Figure 1: Training log showing loss convergence over 50 epochs.

## 3 Manipulation via SDE-Edit

To edit an image, we added noise to the original vector and then denoise it conditioned on the target label (Target=1.0 for Male).

Listing 2: SDE Edit Logic

```
def sde_edit(w_source, target_label, start_t, guidance_scale=2.0):

    #Modifies w_source to match target_label using the diffusion model.

    #Forward Process (Destruction)
    #We add noise to the source vector up to start_t
    #We create a batch of 1 for the start_t

    t_tensor = torch.full((1,), start_t, device=device, dtype=torch.long)

    noisy_w, _ = get_noisy_image(w_source, t_tensor)

    #Reverse Process (Reconstruction)
    current_w = noisy_w

    #We loop backwards from start_t to 0
    for t in tqdm(reversed(range(0, start_t)), desc="Sampling", total=start_t):
        t_tensor = torch.full((1,), t, device=device, dtype=torch.long)
        t_input = (t_tensor.float() / num_timesteps).unsqueeze(1)

        #Classifier-Free Guidance
        #We run the model twice: once with the label, once without
        c_target = torch.full((1, 1), target_label, device=device).float()
        c_null = torch.zeros((1, 1), device=device).float()

        # Concatenate for efficiency: [cond_input, null_input]
        w_in = torch.cat([current_w, current_w])
        t_in = torch.cat([t_input, t_input])
        c_in = torch.cat([c_target, c_null])

        noise_pred_chunk = model(w_in, t_in, c_in)
        noise_cond, noise_uncond = noise_pred_chunk.chunk(2)

        #Apply Guidance: pred = uncond + scale * (cond - uncond)
        noise_pred = noise_uncond + guidance_scale * (noise_cond - noise_uncond
            )

        #Denoising Step
        alpha = alphas[t]
        alpha_cumprod = alphas_cumprod[t]
        beta = betas[t]

        if t > 0:
            noise = torch.randn_like(current_w)
        else:
            noise = torch.zeros_like(current_w)

        term1 = 1 / torch.sqrt(alpha)
        term2 = (1 - alpha) / torch.sqrt(1 - alpha_cumprod)

        current_w = term1 * (current_w - term2 * noise_pred) + torch.sqrt(beta)
             * noise

    return current_w
```

# 4 Experimental Results

We conducted a grid search to analyze the effect of Starting Noise ($t$) and Guidance Scale ($s$).

## 4.1 Noise vs. Guidance Grid

The figure below shows the source image (Female) manipulated towards the target (Male).


Figure 2: Grid Analysis. **Rows:** Starting Noise levels ($t = 50, 150, 300$). **Columns:** Guidance Scales ($s = 1.0, 5.0, 10.0$).

## 4.2 Analysis

- **Effect of Noise ($t$):** At low noise, the face changes and the identity is preserved, but the gender barely changes. The masculine features such as beard or sharper facial structure does not included to the manipulation. At high noise, the gender swap is successful, but the original identity is completely lost, moreover with a higher guidance the image corrupts.
- **Effect of Guidance ($s$):** Increasing guidance forces the model to generate stronger male-like features, adding facial hair and changing the hairline. At lower noise levels this changes is limited.

# 5 Discussion

## 5.1 Latent Diffusion vs. Pixel Diffusion

We trained on $\mathcal{W}$ vectors (size 512) rather than pixels ($1024 \times 1024 \times 3$).

- **Computational Advantage:** The dimensionality is reduced by a factor of $\approx 6000\times$. This allows us to train the diffusion model faster than the pixel-based diffusion.
- **Semantic Richness:** $\mathcal{W}$ space is already semantically organized by StyleGAN. Diffusion in this space moves along semantic manifolds rather than just changing pixel values.

## 5.2 The Trade-off: Noise vs. Identity

We observed a fundamental trade-off. To change a high-level attribute like gender, we must destroy enough information in the original vector, in other words, we need to add higher noise to allow the model to "hallucinate" new features. However, destroying this information removes the constraints that hold the original identity (facial structure, background, pose of the figure etc.). This explains why the bottom row of Figure 2 looks like a stranger.

## 5.3 Project 1 (Linear) vs. Project 2 (Diffusion)

- **Linear Interpolation:** Moves in a straight line. It is fast but assumes the manifold is flat. It often caused "entanglement" (e.g., changing background brightness when adding a smile).
- **Diffusion:** Follows a non-linear trajectory via the reverse SDE. It produces more realistic results by staying strictly within the distribution of valid faces. However, it is harder to control identity preservation compared to linear editing.

# 6 Failure Case Analysis


Figure 3: Failure Case ($t = 300, s = 10.0$). The manipulation failed to preserve identity. The noise level was so high that the source signal was drowned out, causing the model to generate a monstrous face rather than modifying the original female face.